ARIADNA NÚÑEZ TORRES 8 de octubre de 2026



Se ingresa a la pagina de Spark para poder tenerla en el momento de su descarga, pero no sin antes tener que descargar los demás componentes que necesita esta herramienta para su debido

Java SE Development Kit 21.0.12.1 downloads

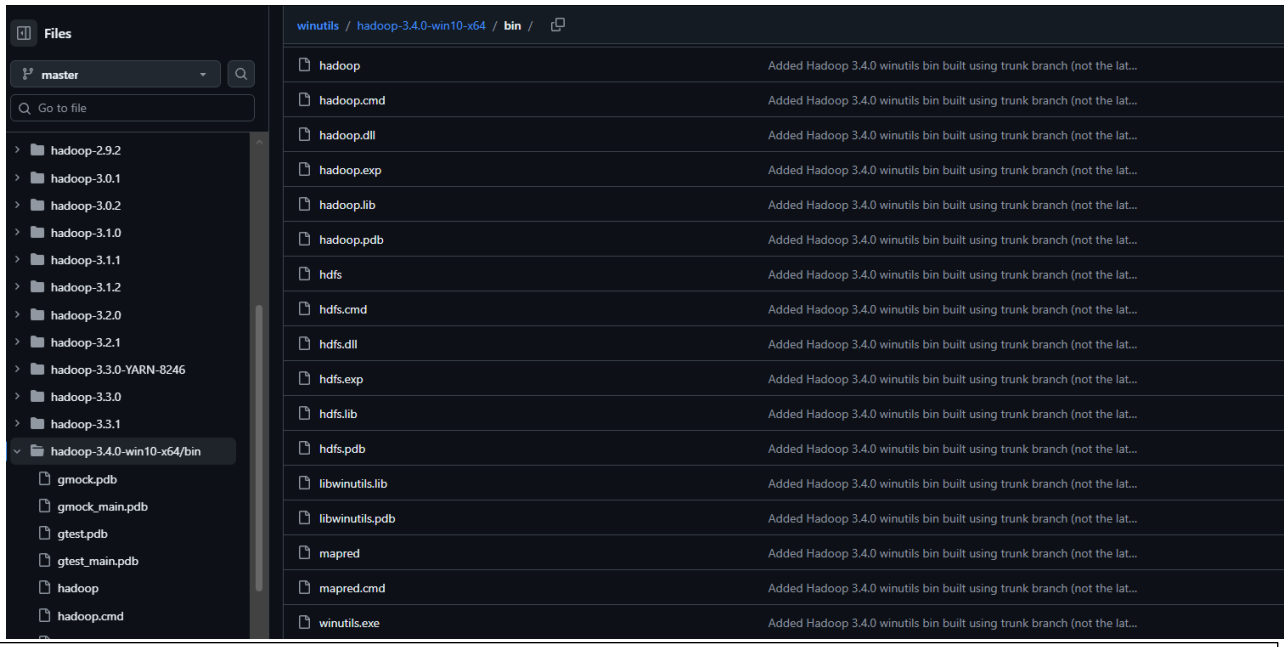
JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the SE OTN License (OTN) and production use beyond the limited free grants of the OTN license will require a fee.

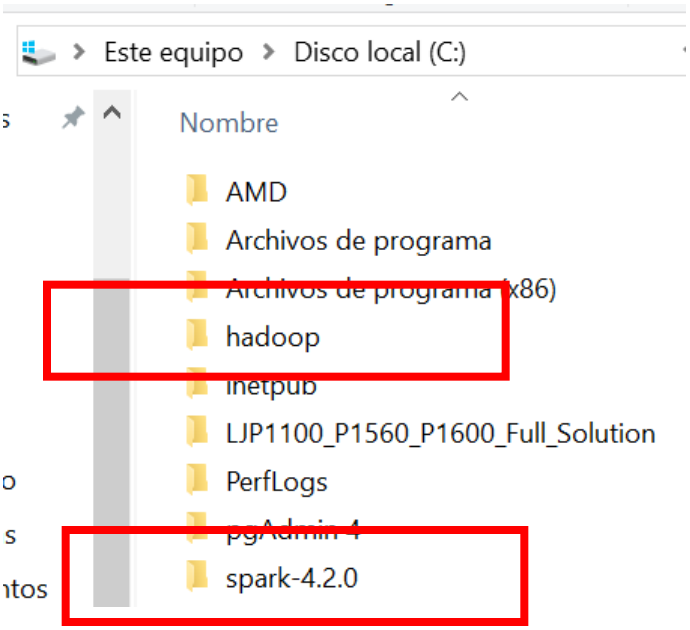
Linux **macOS** **Windows**

Product/file description	File size	Download
x64 Compressed Archive	187.82 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	166.94 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	165.71 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

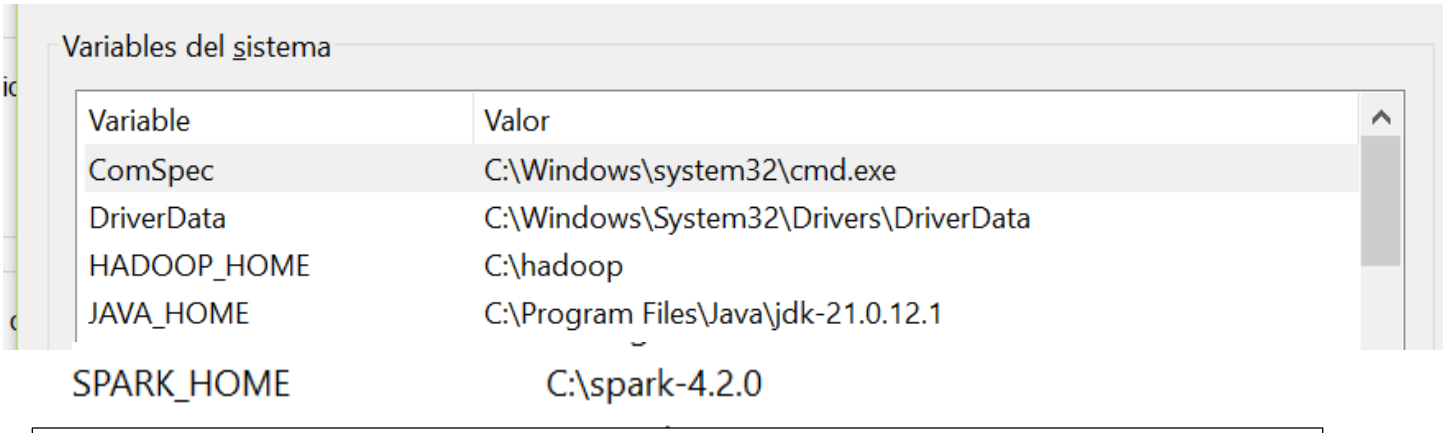
Su primer componente a requerir es Java en SE 21, su instalador será el de MSI que permitirá el uso de este ***enviroment*** dentro de VS Studio. Otro componente que se solicitó fue Python pero ese lenguaje ya se instalo previamente



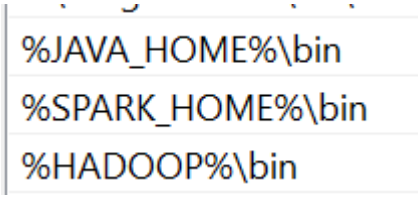
Como siguiente y previo a instalar Spark se obtendrá el ejecutable de hadoop, al tenerlo se instalará junto con Spark en su última versión con ello se pasar a la parte de unas modificaciones para que al momento de editar e insertar variables no haya errores



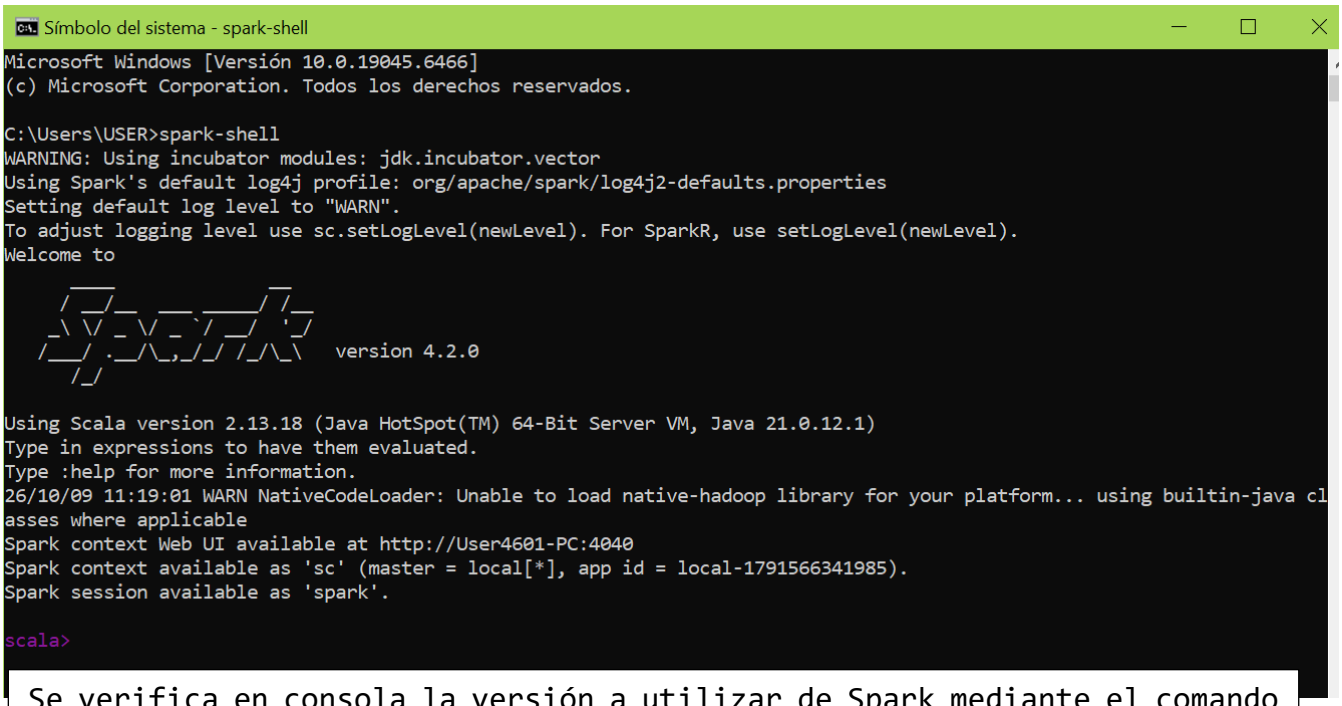
Ambos instalados en sus ultimas versiones asi como hadoop con su respectivo bin se procederá a siguientes pasos en el entorno de variables



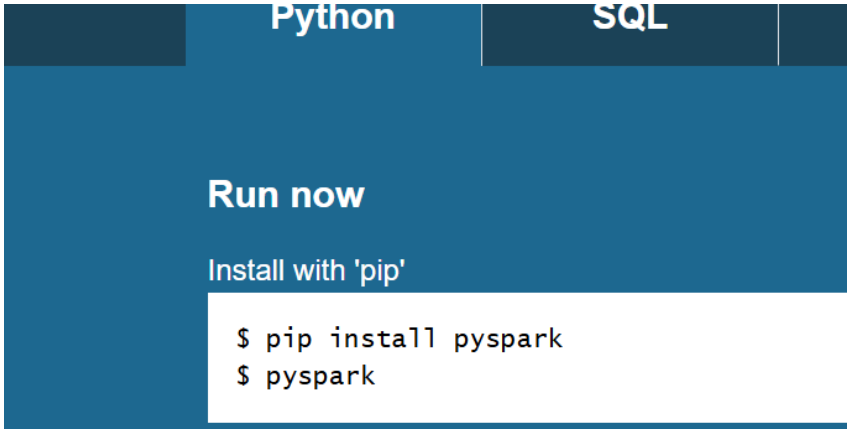
Dentro del entorno de variables se crean dos que tengan el directorio de los archivos creados, pasando de ese se procederá a crear dos tres nuevos entornos dentro de ***PATH***



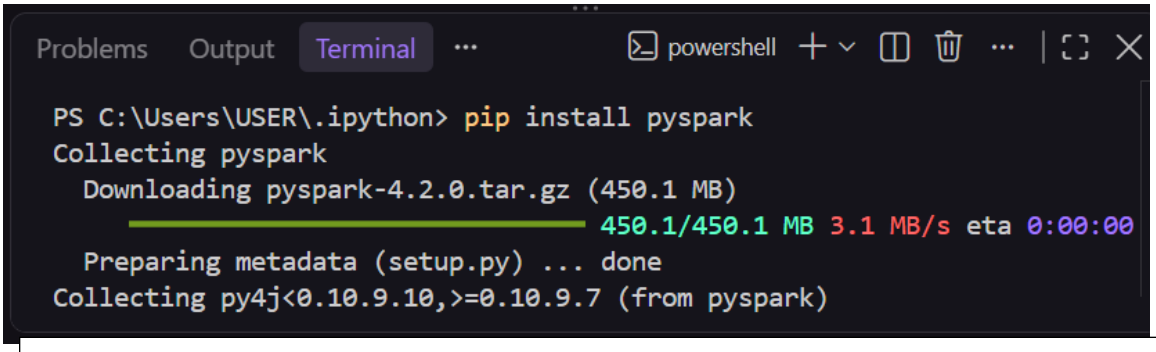
Estas tres nuevas variables son las que se van a crear para la instalación y verificación de Spark en CMD



Se verifica en consola la versión a utilizar de Spark mediante el comando de ***spark – Shell*** permitiéndole el acceso a Java, todo esto para que el entorno al momento de trabajar con este sea permitido



De paso se instala la parte de PySpark que es con la que se trabajará en el entorno de Anaconda



En el entorno de VS Code se realiza la instalación del pip de la extensión de Spark con Python

CONFIGURACIÓN DE SPARK

Dentro de este ejercicio se revisara la configuración básica de Spark y la diferencia con las bases de datos tradicionales

Se realizará principalmente la instalación, selección y agrupamiento de datos para posteriormente su ordenamiento

Configuración de Plataforma Spark

```
#Se importan librerias de spark para realizar el dataframe
from pyspark.sql import SparkSession
```

✓ 0.0s

```
#Se crea cliente de spark
spark = SparkSession \
        .builder \
        .appName('LecturaArchivoCSV') \
        .getOrCreate()
```

Al tener la configuración de la herramienta spark se comienza a obtener las librerías a utilizar en este caso para poder trabajar con el, usando ***SparkSession***

Importacion de datos

```
#Se importa el archivo a utilizar
path_archivo = 'databses/kc_house_data.csv'
house = spark.read.csv(path_archivo, header= True)
```

✓ 0.2s

```
#Se visualiza el dataframe creado
house.show()
```

✓ 0.1s

```
+----------+---------------+----------+--------+---------+-----------+--------+------+----------+----+--
|        id|           date|     price|bedrooms|bathrooms|sqft_living|sqft_lot|floors|waterfront|view|co
+----------+---------------+----------+--------+---------+-----------+--------+------+----------+----+--
|7129300520|20141013T000000|    221900|       3|        1|       1180|    5650|     1|         0|   0|
|6414100192|20141209T000000|    538000|       3|     2.25|       2570|    7242|     2|         0|   0|
|5631500400|20150225T000000|    180000|       2|        1|        770|   10000|     1|         0|   0|
|2487200875|20141209T000000|    604000|       4|        3|       1960|    5000|     1|         0|   0|
|1954400510|20150218T000000|    510000|       3|        2|       1680|    8080|     1|         0|   0|
|7237550310|20140512T000000|1.225e+006|       4|      4.5|       5420|  101930|     1|         0|   0|
```

Se importa el archivo a utilizar que sería casi como utilizar ***Pandas*** pero aquí se tiene que especificar el header con la funcion ***Header***

```
#Ubicar el tipo de dato que contiene el dataframe
house.dtypes
```

✓ 0.0s

```
[('id', 'string'),
 ('date', 'string'),
 ('price', 'string'),
 ('bedrooms', 'string'),
 ('bathrooms', 'string'),
 ('sqft_living', 'string'),
 ('sqft_lot', 'string'),
 ('floors', 'string'),
 ('waterfront', 'string'),
 ('view', 'string'),
 ('condition', 'string'),
 ('grade', 'string'),
 ('sqft_above', 'string'),
```

Para siguiente paso se verifica el tipo de dato que tiene cada columna, en pasos siguientes se hará cambio de estos

```
#Se cambia el tipo de dato para que no tenga errores
import pyspark.sql.functions as F
house = (house
    .withColumn('price',F.col('price').cast('double'))
    .withColumn('bedrooms',F.col('bedrooms').cast('double'))
    .withColumn('bathrooms',F.col('bathrooms').cast('double'))
    .withColumn('zipcode',F.col('zipcode').cast('int'))
    .withColumn('lat',F.col('lat').cast('double'))
    .withColumn('long',F.col('long').cast('double'))
)
```

En este paso es donde se hará el cambio de tipo de dato ya que si no se realiza podría llegar a tener varios problemas con distintas funciones ya sea con python o con SQL

Selección de datos con filtros sencillos

```
#Listado completo de columnas ordenado por zipcode
import pyspark.sql.functions as F1
house.selectExpr('try_cast(zipcode as int)')
house.sort(F1.col('zipcode')).show(10)
```

✓ 0.4s

```
+----------+---------------+--------+--------+---------+-----------+--------+------+----------+----+---------+-----+
|        id|           date|   price|bedrooms|bathrooms|sqft_living|sqft_lot|floors|waterfront|view|condition|grade|
+----------+---------------+--------+--------+---------+-----------+--------+------+----------+----+---------+-----+
|8159610030|20140722T000000|196000.0|     3.0|     2.25|       2070|   11576|     2|         0|   0|        3|    7|
|3751600030|20140717T000000|100000.0|     2.0|      1.0|        770|   17334|     1|         0|   0|        3|    7|
|3329500060|20140728T000000|305000.0|     4.0|      2.5|       2250|    9091|     1|         0|   0|        3|    7|
|3353401710|20140923T000000|227950.0|     3.0|      1.5|       1670|    8230|     1|         0|   0|        5|    7|
|7967200060|20140908T000000|243000.0|     3.0|     1.75|       1450|   12125|     1|         0|   0|        4|    7|
|3717000160|20141009T000000|287000.0|     4.0|      2.5|       2240|    4648|     2|         0|   0|        3|    7|
```

Un filtro sencillo que se realizara es el ordenamiento del dataframe a base del zipcode que para poder hacer esto se importa F1 que es como se puede ordenar base a una columna en especifico

```
#Zipcode con mayor numero de casas, calculo de promedio de precio, tamaño en m2
house.createOrReplaceTempView('HOUSES')
#Se ejecuta la consulta tipo sql
sql_str = 'select zipcode, avg(price), lat * long as Metros_Cuadrados from HOUSES group by zipcode, price,Metros_Cuadrados'
spark.sql(sql_str).show()
```

✓ 1.2s

```
+-------+----------+------------------+
|zipcode|avg(price)|   Metros_Cuadrados|
+-------+----------+------------------+
|  98133|  420000.0|      -5837.9045544|
|  98133|  371500.0| -5840.791459800001|
|  98045|  415000.0|      -5776.5987976|
|  98070|  369900.0|      -5811.9237154|
|  98024| 1550000.0|        -5804.25903|
|  98112|  490000.0|      -5823.8757792|
|  98108|  442000.0|      -5815.9125122|
```

Como siguiente paso el que se realizará será en base a una tabla temporal de SQL, esto para realizar una consulta de columnas agrupadas con cierto tipo de funciones agregadas en especifico de unas columnas como lo son ***lat, long*** para los metros cuadrados***, y Price*** para sacar precio promedio como lo solicitado

Agrupamientos

```
#Se agrupa por numero de habitaciones, baños y precio promedio por zipcode
house.groupBy(['zipcode','bedrooms','bathrooms']).avg('Price').show()
```

16] ✓ 0.5s

```
+-------+--------+---------+-----------------+
|zipcode|bedrooms|bathrooms|       avg(Price)|
+-------+--------+---------+-----------------+
|  98023|     4.0|      2.5|327764.6551724138|
|  98117|     3.0|      3.5|         608250.0|
|  98072|     3.0|      2.0|         505125.0|
|  98144|     3.0|      2.5|         542555.0|
|  98023|     5.0|     1.75|         318250.0|
|  98056|     4.0|      3.5|637983.3333333334|
|  98056|     2.0|      1.0|239543.16666666666|
|  98133|     4.0|     2.75|         441686.25|
```

Por ultimo otra agrupación sin ser una consulta de SQL donde será con base a tener el precio promedio el numero de baños, habitaciones de las casas